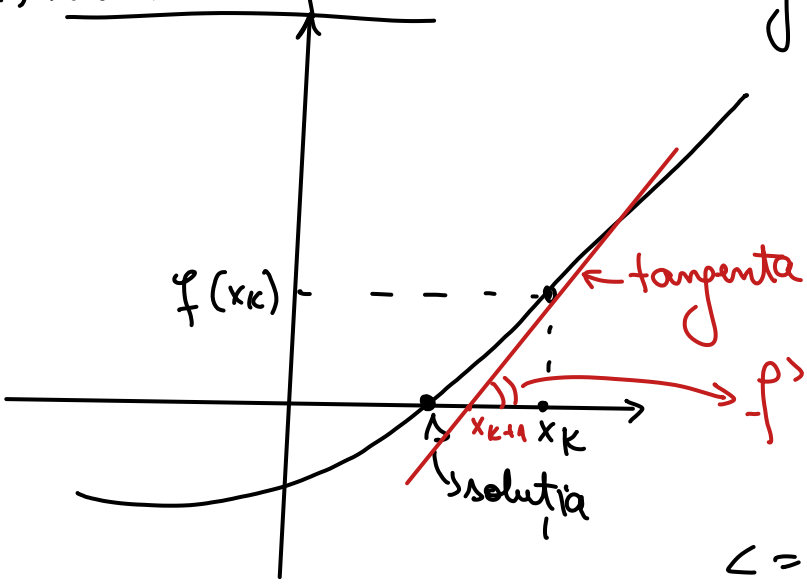
Ec neliniare

1) Newton-Raphson (metoda tangentei)



$$f'(x_k) = \frac{y_2 - y_1}{x_2 - x_1} = \frac{0 - f(x_k)}{x_{k+1} - x_k}$$

$$\Leftrightarrow f'(x_k)(x_{k+1} - x_k) = -f(x_k)$$

$$\Leftrightarrow x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

until err e acceptabilă

Prop

→ șirul nu converge când $f'(x_n) = 0$ sau $|x_{k-1} - x_k| > 1$

Avantaj → e optimă pt că are ordin de conv $R = 2$ $O(n^2)$

$n + 1 = 2 + 1 = 3$ ↳ nr de apeluri

$R \leq n + 1$

$$\lim \frac{\varepsilon_{k+1}}{|\varepsilon_k|^R} = A \in \mathbb{R}$$

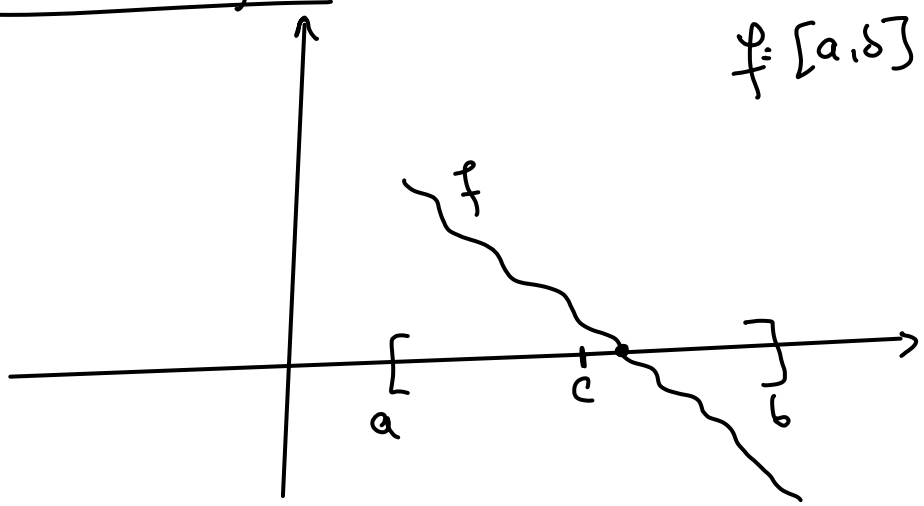
→ converge în 3-5 pași pt un x_0 bine ales (cu bisectie găsit usually)

Dez → fie există f'

→ 2 apeluri de f

→ uneori nu converge

2) Metoda bisectiei



$f: [a,b]$

$c = \frac{a+b}{2}$

$f(b) \cdot f(c) < 0$

$a = c$, $c = \frac{a+b}{2}$

$f(a) \cdot f(c) < 0$

$b = c$, $c = \frac{a+b}{2}$

condiție $|b-a| < tol$

Norme ale matrici → Frobenius $\sqrt{Tr(AA^*)}$

→ max(suma el în modul pe fiecare linie)

Dezavantaje → converge mai prost

ex: Aproximează $\sqrt{3}$ cu 2 zecimale $\Rightarrow tol = 10^{-2}$

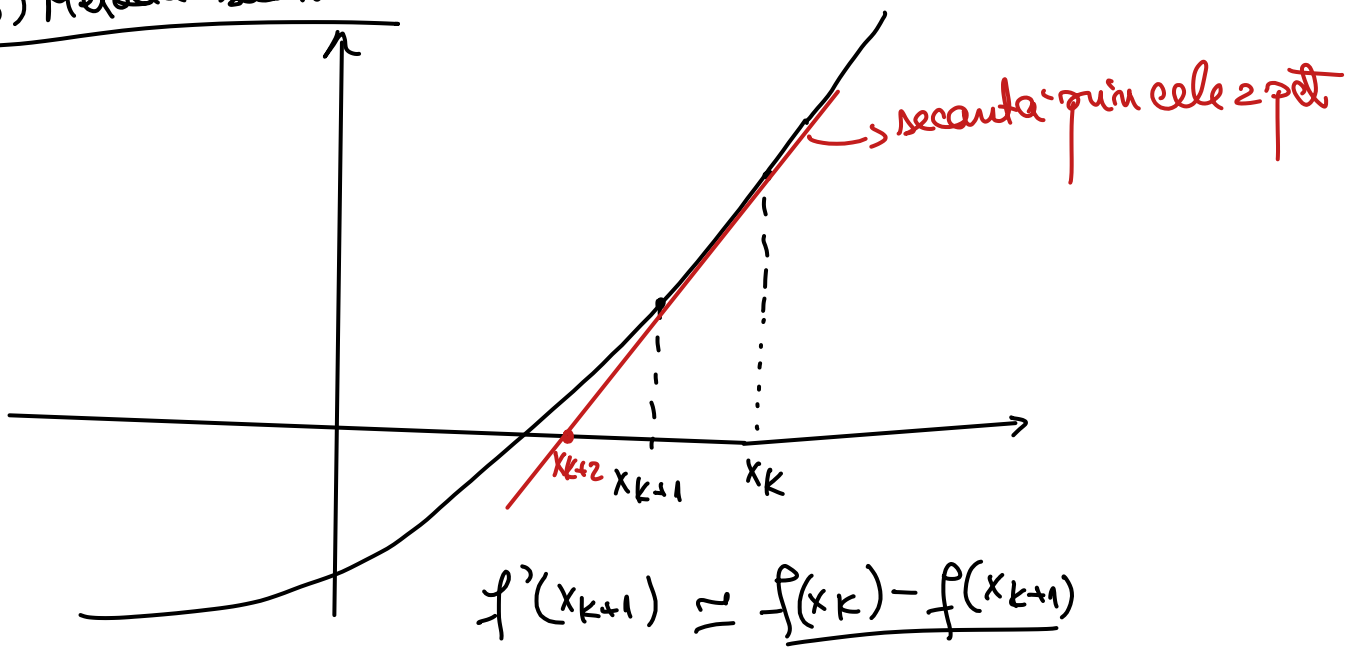
fie $f(x) = x^2 - 3$

$f'(x) = 2x$

fie $x_0 = 2 \Rightarrow x_1 = x_0 - \frac{f(x_0)}{f'(x_0)} = 2 - \frac{1}{4} = \frac{7}{4} = 1,75$ $\quad tol\ 10^{-1}$

$x_2 = x_1 - \frac{f(x_1)}{f'(x_1)} = 1,75 - \frac{- - -}{\dots} = \underline{1,73}$

3) Metoda secantei



$$f'(x_{k+1}) \simeq \frac{f(x_k) - f(x_{k+1})}{x_k - x_{k+1}}$$

înlocuiești în Newton - Raphson

$$\Rightarrow x_{k+2} = x_{k+1} - \frac{f(x_{k+1})}{\frac{f(x_k) - f(x_{k+1})}{x_k - x_{k+1}}} = \frac{(x_k - x_{k+1}) f(x_{k+1})}{f(x_k) - f(x_{k+1})}$$

⊕ scapi de derivată

⊖ converge mai lent

Sisteme de ec neliniare $\Rightarrow$ M. Newton

$$\begin{cases} f_1(x_1, \dots x_n) = 0 \\ f_2(x_1, \dots x_n) = 0 \\ \dots \\ f_n(x_1 \dots x_n) = 0 \end{cases} \quad \Leftrightarrow F(x) \qquad X = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}$$

$$F = \{f_1 \dots f_n\}$$

$$J(F) = \begin{pmatrix} \frac{df_1}{dx_1} & \frac{df_1}{dx_2} & \dots & \frac{df_1}{dx_n} \\ - & - & - & - \\ \frac{df_n}{dx_1} & - & - & \frac{df_n}{dx_n} \end{pmatrix}$$

$$x_{n+1} = x_n - J^{-1}(x_n) F(x_n)$$

Dar J^{-1} e greu de calculat

$$x_{n+1} - x_n = -J^{-1}(x_n) F(x_n)$$

$$\Leftrightarrow J(x_n) \cdot \Delta x_n = -J^{-1}(x_n) F(x_n)$$

$$\boxed{J(x_n)(x_{n+1} - x_n) = -F(x_n)}$$

⊕ Converge rapid

⊕ $O(n^3)$

⊖ J^{-1} — Δx_n — interpolare

dar poate fi omis

Pct fixe $f: D \to \mathbb{R}$ are un sg pct fix pe $D \Leftrightarrow$

$$\Leftrightarrow \begin{cases} \varphi \text{ deriv pe } D \\ |\varphi'(x)| < K < 1 \end{cases}$$

5. Considerați un sistem de 2 ecuații neliniare cu 2 necunoscute pt care alegeți un domeniu D. Asigurați-vă că sistemul are un unic punct fix în D. Rezolvați sistemul iterativ și eficient*, pornind de la o soluție inițială din D și opriți-vă după 2 pași.

ex

$$\begin{cases} \varphi_1(x,y) = \frac{x}{2} - \frac{\sin(x+y)}{3} \\ \varphi_2(x,y) = \frac{x}{2} + \frac{\cos(x+y)}{4} \end{cases}$$

Fie $D = [0,1]$

$$\frac{df_1}{dx} = \frac{1}{2} - \frac{\cos(x+y)}{3} \le \frac{5}{6} < 1 \qquad \frac{df_2}{dx} = \frac{1}{2} - \frac{\sin(x+y)}{4} < 1$$

$$\frac{df_1}{dy} = -\frac{\cos(x+y)}{3} \Rightarrow |\dots| < 1 \qquad \frac{df_2}{dy} = -\frac{\sin(x+y)}{4} \Rightarrow |\dots| < 1$$

Luăm $s = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$ $(x_0 = 0, y_0 = 0)$

I. $J(s_0)(s_1 - s_0) = -F(s_0)$

$$\begin{pmatrix} \frac{1}{6} & -\frac{1}{3} \\ \frac{1}{2} & 0 \end{pmatrix} \begin{pmatrix} x_1 \\ y_1 \end{pmatrix} = -\begin{pmatrix} 0 \\ \frac{1}{4} \end{pmatrix}$$

$$\Leftrightarrow \begin{cases} \frac{x_1}{6} - \frac{y_1}{3} = 0 \\ \frac{x_1}{2} = -\frac{1}{4} \end{cases} \Leftrightarrow \begin{cases} x_1 = 2y_1 \Rightarrow y_1 = -\frac{1}{4} \\ x_1 = -\frac{1}{2} \end{cases}$$

II. $J(s_1)(s_1 - s_0) = -F(s_1)$

.... calcule ...

Sisteme ec. liniare

→ Descompunerea LU { Crout, Doolittle, Cholesky

• nr conditionare $\text{cond}(A) = \|A\| \|A^{-1}\|$

Prop: { $\text{cond}(\alpha A) = \text{cond}(A)$
$\text{cond}(A) \ge 1 \quad \forall A$; $\text{cond} A = 1 \Leftrightarrow A = \alpha I_n$
$\text{cond}(A) = \infty \Rightarrow A$ singulara (= neinversabilă)
A diag $\Leftrightarrow \text{cond}(A) = \max_{a_{ii} \in A} |a_{ii}| \cdot \max_{a_{jj} \in A^{-1}} |a_{jj}|$

bine conditionat → 1

rău conditionat → ∞ (e mai mare)

$\text{cond}(A) = \|A\| \cdot \|A^{-1}\|$ → cât de aprox

$Ax = b$

$\|A^{-1}\| \geq \frac{\|x\|}{\|b\|} \Rightarrow \|A^{-1}\| = \frac{\|x\|}{\|b\|}$

ex $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ iau arbitrar $x = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$

$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 4 \end{bmatrix}$

$1 \qquad \sqrt{20}$

iau norma $\|A\| = \sqrt{\sum a_{ij}^2}$

$\Rightarrow \|A^{-1}\| = \frac{1}{\sqrt{20}} \Rightarrow \text{cond}(A) = \frac{30}{\sqrt{20}}$

7. Care dintre matricile $A = \begin{bmatrix} 0.0003 & 3 \\ 1 & 1 \end{bmatrix}$ și $B = \begin{bmatrix} 1 & 1 \\ 1 & 1.0001 \end{bmatrix}$ este rău condiționată? (număr de condiționare)

iau $x = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \Rightarrow Ax = \begin{bmatrix} 3 \\ 1 \end{bmatrix} \Rightarrow \|A^{-1}\| = \frac{1}{\sqrt{10}}$

$\text{cond}(A) = \|A\| \cdot \|A^{-1}\| \simeq \sqrt{11} \cdot \frac{1}{\sqrt{10}} > 1$

$\begin{bmatrix} 1 & 1 \\ 1 & 1{,}0001 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \Rightarrow \text{cond}(B) \simeq 2 \cdot \frac{1}{\sqrt{2}} = \sqrt{2}$

$\sqrt{1,1} < \sqrt{2} \Rightarrow A$ mai bun cond ca B

- Mat Hessenberg → inferioară = lower + pseudodiag de deasupra
  → superioară = upper + pseudodiag de sub

→ Transformări ortogonale pt QR — Givens
Householder

Teorie SEL

-> iterative ⟶

$A = N - P = D + L + U$

$x_{k+1} = G x_k + c$

1) Jacobi

$$\begin{cases} N = D \\ P = L + U \end{cases}$$

$G_j = N^{-1}P$, $c_j = N^{-1}b$

G_j ↓ matricea de iterație

c_j → vector de iterație

⊕ poate fi paralelizat

⊖ conv mai greu pt că se folosește de pasul anterior

$O(n^3)$

$x_{m+1} = G_j x_m + c_j$

2) Gauss-Seidel

$$\begin{cases} N = D - L \\ P = U \end{cases}$$

$G_{GS} = N^{-1}P$, $c_{GS} = N^{-1}b$

⊕ converge mai repede pt că se fol. și de pasul curent

⊖ nu poate fi paralelizat ușor

⊖ instabilitatea el. calculate anterior

$O(n^3)$

3) SOR

$$\begin{cases} N = D + \omega L \\ P = (1-\omega)D - \omega U \end{cases}$$

$G_{SOR} = N^{-1}P$, $c_{SOR} = \omega N^{-1}b$

⊕ ω bine ales -> conv rapid

⊖ ω prost ales -> conv lent

Forma algebrică: $x_{k+1} = (1-\omega)x_k + \omega G_{G-S}$

Cond pt ↑: $\rho(G) < 1$ (cu cât e mai mic $\rho(G)$, cu atât conv mai repede)

Metoda Gradientului Conjugat

$Ax = b$

Conditii $A \Rightarrow A$= semipozitiv def $\Leftrightarrow x^t A x \geq 0 \ \forall x \neq 0$

altfel nu merge metoda.

Teorema x' sol sist $\Leftrightarrow x'$ minimizează fct

$g(x) = \langle x, Ax \rangle - 2\langle x, b \rangle \Rightarrow g(x') \leq g(x) \ \forall x$

$x_{k+1} = x_k - t_{k+1} \cdot v_{k+1}$ → vector de direcție

↳ coef de iteratie

$t_{k+1} = \frac{\langle v_k, r_k \rangle}{\langle v_k, A v_k \rangle}$ unde $r_k = b - A x_k$

Pt a afla $v \Rightarrow$ MI. Met. pașilor desc

$v_k = -\nabla g(x_k) = r_k$

⊕ e simplu

$\Rightarrow$ ⊖ converge lent ☹

MII. Met direcției conjugate

⊕ induce convergența în n pași

$x_{k+1} = x_k - t_{k+1} \cdot v_{k+1}$

$\rightarrow t_{k+1} = \frac{\langle r_k, r_k \rangle}{\langle v_k, A v_k \rangle}$

$v_k = r_{k-1} + s_{k-1} \cdot v_{k-1}$,

unde $s_{k-1} = \frac{\langle r_{k-2}, r_{k-2} \rangle}{\langle r_{k-1}, r_{k-1} \rangle}$

și $r_{k+1} = r_k - t_k A v_k$

⊖ nu e ușor de paralelizat

12. Ce transformări ortogonale, din cele studiate, se pretează cel mai bine pt a aduce o matrice pătratică Hessenberg în forma superior triunghiulară? Câte transformări Householder ar fi necesare? Câte transformări Givens ar fi necesare?

Cazul inferior: $\begin{cases} H: n-1 \\ G: \frac{(n-1)n}{2} \end{cases}$

Cazul superior $\begin{cases} H: n-1 \\ G: n-1 \end{cases}$

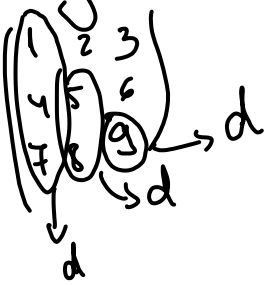
Householder:

$$H_i = I_n - \frac{2vv^t}{v^t v}$$

$$v = \begin{pmatrix} 0 \\ \vdots \\ 0 \\ \sigma + a_{ii} \\ \vdots \\ a_{ni} \end{pmatrix}$$

$$\sigma = -\operatorname{sgn}(a_{ii}) \cdot \|d\|$$

ex



Givens:

$r = \sqrt{x^2 + y^2}$ ← val de pe diag prime

pivotul

$c = \frac{x}{r} \quad s = \frac{-y}{r}$

⊕ mai stabilă

⊖ mai multe iterații

14. Dem. că dacă A și B sunt matrici asemenea atunci au același spectru de valori proprii și că pt orice matrice A avem relația $\rho(A) \leq \|A\|$.

a) $\sigma(A) = \sigma(B)$

b) $\rho(A) \leq \|A\|$

a) $A \sim B \iff A = PBP^{-1} - \lambda I_n \iff A - \lambda I_n = PBP^{-1} - \lambda I_n$

$\lambda I_n = \lambda P P^{-1}$

$\iff A - \lambda I_n = P(B - \lambda I_n)P^{-1} \mid \det$

$\iff \det(A - \lambda I_n) = \det P \cdot \det(B - \lambda I_n) \det P^{-1}$

$\det P^{-1} = \frac{1}{\det P}$

$\Rightarrow$ pol caract sunt identice dacă $A \sim B$

b) $\rho(A) = \max |\lambda_k| = |\lambda_p| \to x_p$

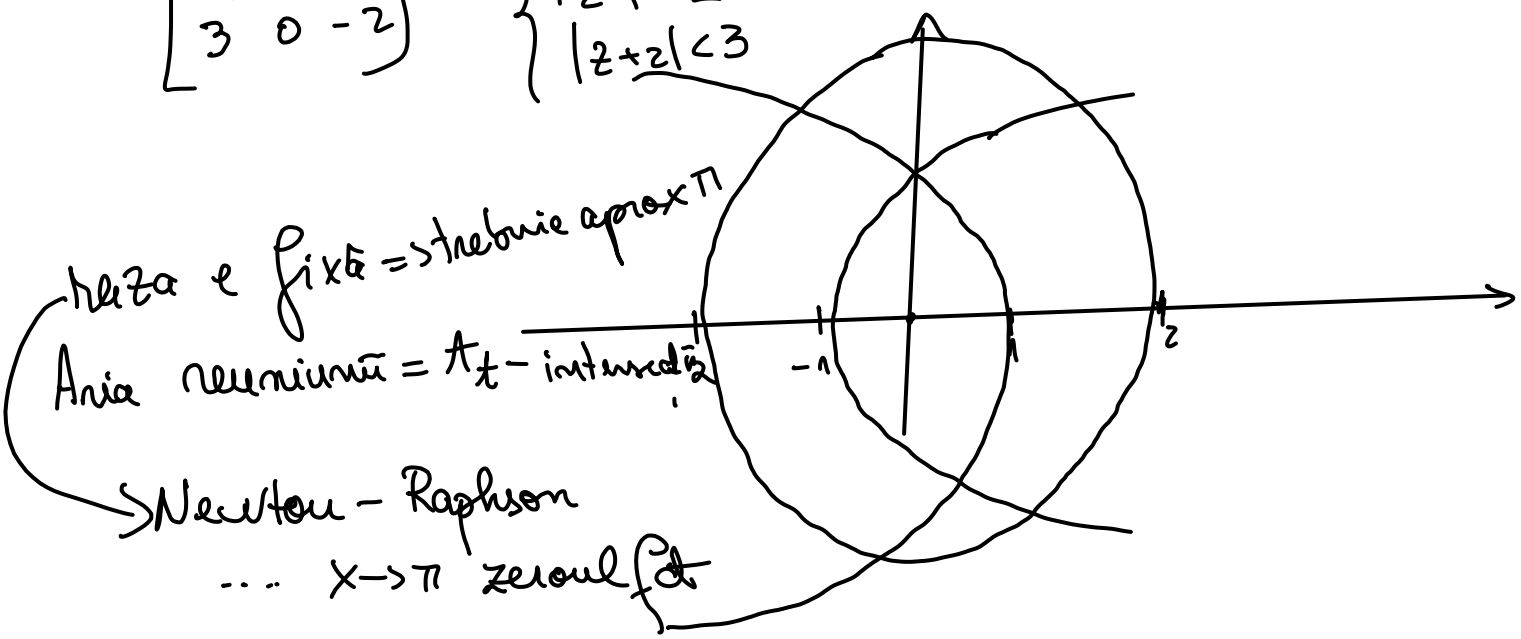
pt că $Ax_p = \lambda_p x_p$

$$\rho(A) = |\lambda_p| = |\lambda_p| \cdot \|x_p\| = \|\lambda_p x_p\| = \|Ax_p\| \le \|A\| \cdot \|x_p\| = \|A\|$$

($\|x_p\| = 1$)

Cercurile lui Gershgorin

$$\begin{bmatrix} 1 & 0 & 2 \\ -1 & 0 & -1 \\ 3 & 0 & -2 \end{bmatrix} \Rightarrow \begin{cases} |z-1| < 2 \\ |z| < 2 \\ |z+2| < 3 \end{cases}$$



Rădăcina e fixă => trebuie aprox π

Aria reuniunii = A_t - intersecții

Newton - Raphson

.... $x \to \pi$ zeroul fct

1. Considerăm $A = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{bmatrix}$ Localizați valorile submatricei $A[\{1,2,3\}, \{1,2,4\}]$

cu ajutorul discurilor Gershgorin.

păstrăm liniile 1,2,3 și col 1,2,4

$$\Rightarrow M = \begin{bmatrix} 1 & 2 & 4 \\ 5 & 6 & 8 \\ 9 & 10 & 12 \end{bmatrix} \Rightarrow \begin{array}{l} C_1\ |z-1| < 6 \Rightarrow z \in (-5, 7) \\ C_2\ |z-6| < 13 \Rightarrow z \in (-7, 19) \\ C_3\ |z-12| < 19 \Rightarrow z \in (-7, 31) \\ \quad \Leftrightarrow -19 < z-12 < 19 \end{array}$$

$\Rightarrow C_1 \cup C_2 \cup C_3 = C_3 \Rightarrow$ Toate valorile proprii se găsesc în C_3

$\Rightarrow \lambda = a + bi,\ a, b \in \mathbb{R} \Rightarrow a \in (-7, 31),\ b \in (-19, 19)$

2. Rezolvați eficient d.p.d.v al implementării, prin eliminare gaussiană, următorul sistem de ecuații liniare $A = \begin{bmatrix} 3 & 2 & 102 \\ 2 & -3 & 101 \\ 1 & 1 & 2 \end{bmatrix}$, $b = \begin{bmatrix} 101 \\ 96 \\ 2 \end{bmatrix}$. (scalare)

$$\left[\begin{array}{ccc|c} 3 & 2 & 102 & 101 \\ 2 & -3 & 101 & 96 \\ 1 & 1 & 2 & 2 \end{array}\right]$$

I. $\frac{3}{102} > \frac{2}{101} < \frac{1}{2}$

$L_1 \leftrightarrow L_3$ $\begin{bmatrix} 1 & 1 & 2 & : & 2 \\ 2 & -3 & 101 & : & 96 \\ 3 & 2 & 102 & : & 101 \end{bmatrix}$ $\begin{matrix} -2L_1+L_2 \\ -3L_1+L_3 \end{matrix}$ $\begin{bmatrix} 1 & 1 & 2 & : & 2 \\ 0 & -5 & 97 & : & 92 \\ 0 & -1 & 96 & : & 95 \end{bmatrix}$

II. $\frac{5}{97} > \frac{1}{2}$ $\Rightarrow$ rămân la fel L_2 și L_3

$-\frac{1}{5}L_2+L_3$ $\begin{bmatrix} 1 & 1 & 2 & : & 2 \\ 0 & -5 & 97 & : & 92 \\ 0 & 0 & -\frac{97}{5}+96 & : & -\frac{92}{5}+95 \end{bmatrix}$

$\Rightarrow \left(-\frac{97}{5}+96\right)x_3 = -\frac{92}{5}+95 \mid \cdot 5$

$\Leftrightarrow (-97+480)x_3 = -92+475$

$\Leftrightarrow 383 x_3 = 383 \Rightarrow x_3 = 1$

$\Rightarrow -5x_2 + 97x_3 = 92 \Leftrightarrow -5x_2 = -5 \Leftrightarrow x_2 = 1$

$\Rightarrow x_1 + x_2 + 2x_3 = 2 \Leftrightarrow x_1 = -1$

3. Prezentați conceptul de Număr de Condiționare pentru SEL. Explicați detaliat construcția și semnificația.

Numărul de condiționare pt un sistem de ec. liniare se referă la cât de aproape este matricea sistemului de a fi singulară, dar acesta redă și modul în care se schimbă soluțiile sistemului (A) pt mici modificări ale matricei sistemului și a vectorului rezultat (b): o mat. A cu nr cond mic este „bine" condiționată, deci sol. nu se schimbă drastic; nr cond mare $\Rightarrow$ „rău" cond. Prin convenție, dacă mat A sg $\Rightarrow$ nr condiționare $= \infty$

Prop
$$\begin{cases} \text{cond}(A) = \|A\| \cdot \|A^{-1}\| \geq 1 \quad (=1 \text{ pt } A = I_n) \\ \text{cond}(\alpha A) = \text{cond}(A) \\ \text{pt } A \text{ diagonală } \text{cond}(A) = \max(|a_{ii}|, |a_{jj}|) \end{cases}$$
$a_{ii} \in A$, $a_{jj} \in A^{-1}$

De obicei deoarece A^{-1} este greu de calculat ne folosim de sistemul $Ax=b$

$Ax=b \Leftrightarrow x = A^{-1}b \Leftrightarrow \|x\| = \|A^{-1}b\| \leq \|A^{-1}\| \cdot \|b\| \Leftrightarrow \|A^{-1}\| \geq \frac{\|x\|}{\|b\|}$

astfel aflând $\|A^{-1}\|$ fără a mai fi nevoie de A^{-1}

4. Enumerați patru caracteristici ale zerourilor familiilor de polinoame ortogonale.

(Legendre + Cebîșev)

→ reale și distincte

→ pe intervale specifice ceea ce ajută la caract de ortogonalitate $[-1,1]$

→ distribuite simetric în interval

5. Considerați un sistem de 2 ecuații neliniare cu 2 necunoscute pt care alegeți un domeniu D. Asigurați-vă că sistemul are un unic punct fix în D. Rezolvați sistemul iterativ și eficient*, pornind de la o soluție inițială din D și opriți-vă după 2 pași.

Pct fixe $f: D \to \mathbb{R}$ are un sg pct fix pe D $\iff$

$\iff$ $\begin{cases} f \text{ deriv pe } D \\ |f'(x)| < K < 1 \end{cases}$

0

fie $\begin{cases} f_1(x,y) = x - \frac{\sin(x+y)}{5} = 0 \\ f_2(x,y) = x + \frac{\cos(x+y)}{5} = 0 \end{cases}$, fie $D = [0,1]$

$\frac{\partial f_1}{\partial x} = \frac{1-\cos(x+y)}{5} \Rightarrow | \ | \leq \frac{2}{5} < 1$

$\frac{\partial f_1}{\partial y} = \frac{-\cos(x+y)}{5} \Rightarrow | \ | \leq \frac{1}{5} < 1$

$\frac{\partial f_2}{\partial x} = \frac{1-\sin(x+y)}{5} \Rightarrow | \ | \leq \frac{2}{5} < 1$

$\frac{\partial f_2}{\partial y} = \frac{-\sin(x+y)}{5} \Rightarrow | \ | \leq \frac{1}{5} < 1$

$\Downarrow$

sistemul are un sg pct fix pe D

fie $x_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$

$\Rightarrow$ **Metoda Newton:** $x_{n+1} = x_n - J^{-1}(x_n) F(x_n)$

$\iff J(x_n)(x_{n+1} - x_n) = -J^{-1}(x_n) F(x_n)$

$\iff J(x_n)(x_{n+1} - x_n) = -F(x_n)$

$\hookrightarrow \begin{bmatrix} x_{n+1,1} \\ x_{n+1,2} \end{bmatrix}$

$\Rightarrow J(x_0) = \begin{bmatrix} 0 & -\frac{1}{5} \\ \frac{1}{5} & 0 \end{bmatrix}$, $F(x_0) = \begin{bmatrix} 0 \\ \frac{1}{5} \end{bmatrix}$

$\begin{bmatrix} 0 & -\frac{1}{5} \\ \frac{1}{5} & 0 \end{bmatrix} \begin{bmatrix} x_{11} \\ x_{12} \end{bmatrix} = -\begin{bmatrix} 0 \\ \frac{1}{5} \end{bmatrix} \Rightarrow x_1 = \begin{bmatrix} x_{11} \\ x_{12} \end{bmatrix} = \begin{bmatrix} -1 \\ 0 \end{bmatrix}$

$$J(x_1)=\begin{bmatrix} \frac{1-\cos(-1)}{5} & -\frac{\cos(-1)}{5} \\ \frac{1-\sin(-1)}{5} & -\frac{\sin(-1)}{5} \end{bmatrix}=\begin{bmatrix} \frac{1-\cos 1}{5} & -\frac{\cos 1}{5} \\ \frac{1+\sin 1}{5} & \frac{\sin 1}{5} \end{bmatrix}$$

$$F(x_1)=\begin{bmatrix} \frac{-1-\sin(-1)}{5} \\ \frac{-1+\cos(-1)}{5} \end{bmatrix}=\begin{bmatrix} \frac{-1+\sin 1}{5} \\ \frac{-1+\cos 1}{5} \end{bmatrix}$$

$$J(x_1)(x_2-x_1)=-F(x_1)$$

$$\Leftrightarrow \begin{bmatrix} \frac{1-\cos 1}{5} & -\frac{\cos 1}{5} \\ \frac{1+\sin 1}{5} & \frac{\sin 1}{5} \end{bmatrix}\begin{bmatrix} x_{21}-x_{11} \\ x_{22}-x_{12} \end{bmatrix}=-\begin{bmatrix} -1+\frac{\sin 1}{5} \\ -1+\frac{\cos 1}{5} \end{bmatrix}$$

$$\Leftrightarrow \begin{bmatrix} \frac{1-\cos 1}{5} & -\frac{\cos 1}{5} \\ \frac{1+\sin 1}{5} & \frac{\sin 1}{5} \end{bmatrix}\begin{bmatrix} x_{21}+1 \\ x_{22} \end{bmatrix}=-\begin{bmatrix} -1+\frac{\sin 1}{5} \\ -1+\frac{\cos 1}{5} \end{bmatrix}$$

$\Rightarrow$ ... se rezolvă sistemul ...

6. Explicați în ce context se folosește SOR și cum se alege ω.

→ convergență patratică

SOR (sau metoda suprarelaxării) este o metodă iterativă de rezolvare a sistemelor de ec. liniare, fiind o variantă îmbunătățită a metodei Gauss-Seidel, prin introducerea coeficientului ω care, dacă este ales bine, duce la o rezolvare mai rapidă a sistemului. ω este o valoare între 0 și 2, însă $\omega \in (0,1)$ nu prea se folosește, fiind chiar o rezolvare mai încetă a sistemului; $\omega = 1$ duce fix la formula de la Gauss-Seidel; $\omega \in (1,2)$ se observă o îmbunătățire, metoda converge rapid.

7. vezi ieri

8. Explicați conceptul de precondiționare.

În rez. SEL pot apărea probleme dacă A (matricea sistemului) este rău condiționată, deci calcularea inversei e mai grea $\Rightarrow$ folosim
(nu cond e mare)

o matrice M asemenea cu A și rescriem sistemul $C^{-1}A\,x = C^{-1}b$ apoi se folosesc alte metode de rez.

se calc mai ușor

9. Fie $A = \begin{bmatrix} 1 & 2 & 99 \\ 4 & 5 & 101 \\ 7 & 8 & 9 \end{bmatrix}$, $b = \begin{bmatrix} 100 \\ 105 \end{bmatrix}$. Rezolvați d.p.d.v al implementării,

prin eliminare gaussiană, urm. SEL $A[\{1,2,4\},\{1,3,4\}]x = b$. (scalare)

Fie $A' = \begin{bmatrix} 1 & 99 \\ 4 & 101 \end{bmatrix}$

$$\left[\begin{array}{cc|c} 1 & 99 & 100 \\ 4 & 101 & 105 \end{array}\right] \xleftrightarrow{L_1 \leftrightarrow L_2} \left[\begin{array}{cc|c} 4 & 101 & 105 \\ 1 & 99 & 100 \end{array}\right] \xrightarrow{-\frac{1}{4}L_1 + L_2}$$

I. $\frac{1}{99} < \frac{4}{101}$

$$\sim \left[\begin{array}{cc|c} 4 & 101 & 105 \\ 0 & -\frac{101}{4} + 99 & -\frac{105}{4} + 100 \end{array}\right]$$

$$\Rightarrow \left(-\frac{101}{4} + 99\right) x_2 = 100 - \frac{105}{4} \quad |\cdot 4 \iff (-101 + 396)x_2 = 400 - 105$$

$$\rightarrow x_2 = 1$$

$$\Rightarrow 4x_1 + 101 x_2 = 105 \Rightarrow x_1 = 1$$

10. De ce avem nevoie de pivotare la eliminarea gaussiană? Cum se poate alege (optim computațional) elementul candidat pe postul de pivot? Care sunt operațiile fundamentale ce stau la baza eliminării gaussiene

Eliminarea gaussiană are probleme în mai multe situații: apar erori de calcul la împărțire; dacă pivoții sunt 0 nu putem împărți la 0.

ex pt matricea $\begin{pmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \\ 6 & 7 & 8 \end{pmatrix}$ nu se pot elimina 3 și 6 din cauza lui 0.

La pivotarea parțială pivotul este elementul cu modulul maxim de pe coloană, iar apoi se swapuiesc liniile a.î. să fie adus pe diagonală.

La pivotarea totală pivotul este el cu modulul maxim din toată submatricea și e adus pe diag principală prin inversări de linii și coloane → ordinea variabilelor se reține într-un vector de permutare $P = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}$ care se permută și el odată cu coloanele.

11. De ce ajută o factorizare LU a matricei coeficienților rezolvarea unui sistem de ecuații liniare?

Fie $Ax = b$ și $A = L \cdot U \Rightarrow LUx = b \Rightarrow \begin{cases} Ly = b \\ Ux = y \end{cases}$

(L = lower, U = upper, y)

sistem inf Δ și se rez de sus în jos

↳ sistem superior triunghiular și se rez de jos în sus

⇒ Rezolvarea e una mult mai simplă

12. Le transformări ortogonale, din cele studiate, se pretează cel mai bine pt a aduce o matrice pătratică Hessenberg în forma superior triunghiulară? Câte transformări Householder ar fi necesare? Câte transformări Givens ar fi necesare?

Și Householder și Givens merg, fiind metode de aduce matricea Hessenberg în f. superior triunghiulară. Avem 2 cazuri, în f. de tipul matricii Hessenberg.

CAZ I. M. H. superioară →

$$\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ 0 & a_{23} & \cdots & a_{3n} \\ \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & a_{n,n-1} & a_{nn} \end{bmatrix}$$

Cu Householder → $n-1$ transf (sunt $n-1$ col unde vrem să facem 0)

Cu Givens → $n-1$ transf (n-1 elemente unde vrem zero (subdiagonala))

Caz II M.H. inferioară → $\begin{bmatrix} a_{11} & a_{12} & 0 & \cdots & 0 \\ a_{21} & a_{22} & a_{23} & \cdots & 0 \\ \vdots & \vdots & & & \vdots \\ a_{n-1,1} & a_{n-1,2} & \cdots & & a_{n-1,n} \end{bmatrix}$

Cu Householder → tot $n-1$ transf (sunt $n-1$ col unde vrem să facem 0)

Cu Givens → $(n-1)+(n-2)+\dots+1 = \frac{(n-1)n}{2}$ transformări

col 1 col 2 col $n-1$

Deci indiferent de felul matricii Hessenberg or fi mai bine să alegem

13. Care este cea mai simplă formă la care poate fi adusă orice matrice prin transformări de asemănare? Prezentați o descompunere studiată pentru această formă.

Forma Schur $A = QUQ^t$

U → matrice upper

Q → mat ortogonală ($Q^tQ = I_n$)

Folosim un alg. de desc. QR (Householder, Givens). Luăm Hh. Se ia v = col pe care vrem zerouri, $d = v + \|v\| \cdot e_1$, cu $e_1 = \begin{pmatrix} -\text{sgn}(\text{pivot}) \\ 0 \\ \vdots \\ 0 \end{pmatrix}$

Apoi $H_i = I_n - 2\frac{dd^t}{d^td}$ și noul $A = H_iA$ și apoi din nou.

$d^td = \|d\|^2$

PA Givens: Fie i linia și j col pe care e el pe care îl vrem 0.

Matricea G va fi $G = \begin{bmatrix} 1 & \cdots & 0 & \cdots & 0 \\ & & \vdots & & \\ 0 & \cdots & c & \cdots & -s \\ & & \vdots & & \\ 0 & \cdots & s & \cdots & c \\ & & \vdots & & \\ 0 & \cdots & 0 & \cdots & 1 \end{bmatrix}$ (i, j)

x = pivotul pe coloana aia

y = el pe care îl vrem 0

$c = \frac{x}{\sqrt{x^2+y^2}}$

$s = \frac{-y}{\sqrt{x^2+y^2}}$

Apoi noul $A = GA$

14. Dem. că dacă A și B sunt matrici asemenea atunci au același spectru de valori proprii și că pt orice matrice A avem relația $\rho(A) \leq \|A\|$.

a) $\sigma(A) = \sigma(B)$

$A \sim B \Rightarrow A = PBP^{-1} \quad | - \lambda I_n$

$\Leftrightarrow A - \lambda I_n = PBP^{-1} - \lambda I_n$

$\Leftrightarrow A - \lambda I_n = PBP^{-1} - \lambda PP^{-1}$

$A - \lambda I_n = P(BP^{-1} - \lambda P^{-1})$

$A - \lambda I_n = P(B - \lambda I_n)P^{-1} \quad | \det$

$\Leftrightarrow \det(A - \lambda I_n) = \det(P(B - \lambda I_n)P^{-1})$

$= \det P \cdot \det(B - \lambda I_n) \cdot \det(P^{-1})$

$\Leftrightarrow \det(A - \lambda I_n) = \det(B - \lambda I_n)$ ($\det P \cdot \det P^{-1} = 1$)

$\Rightarrow$ A și B au pol. caracteristice egale deci au același spectru

b) $\forall A \quad \rho(A) \leq \|A\|$

Fie $\rho(A) = |\lambda_p|$ val proprie cu v_p = vector propriu aferent $\hookrightarrow \|v_p\| = 1$

$\Rightarrow \lambda_p v_p = Av_p$

$\rho(A) = |\lambda_p| = |\lambda_p| \cdot \|v_p\| = \|\lambda_p \cdot v_p\| = \|A \cdot v_p\| \leq \|A\| \cdot \|v_p\| = 1$

$\leq \|A\|$

15. Pt o mat 4×4 pot aplica transformatele Givens G_{21} și G_{43} în paralel? Care va fi matricea considerată după aplicarea acestor transformate?

Se pot aplica în paralel deoarece implică linii și coloane distincte. Noua matrice $A' = G_{43}G_{21}A$

11	12	13	14
21	22	23	24
31	32	33	34
41	42	43	44

G_{21}: $c_1 = \frac{a_{11}}{\sqrt{a_{11}^2 + a_{21}^2}}$ $\quad s_1 = \frac{-a_{21}}{\sqrt{a_{11}^2 + a_{21}^2}}$

G_{43}: $s_1 = \frac{a_{33}}{\sqrt{a_{33}^2 + a_{43}^2}}$ $\quad s_2 = \frac{-a_{43}}{\sqrt{a_{33}^2 + a_{43}^2}}$

16. Considerați un sist. de 3 ec neliniare cu 3 necunoscute. Rezolvați sistemul iterativ și eficient, cu metoda lui Newton, pornind de la o soluție inițială și opriți-vă după o iterație.

fie $\begin{cases} f_1(x,y,z) = x^2+y^2+z^2-1=0 \\ f_2(x,y,z) = x+y-z=0 \\ f_3(x,y,z) = x^2-y=0 \end{cases}$ $\Rightarrow F(s) = \begin{bmatrix} x^2+y^2+z^2-1 \\ x+y-z \\ x^2-y \end{bmatrix}$

$s = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$

$J(s) = \begin{bmatrix} 2x & 2y & 2z \\ 1 & 1 & -1 \\ 2x & -1 & 0 \end{bmatrix}$ $\quad s_{n+1} = s_n - J^{-1}(s_n) F(s_n)$

fie $s_0 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \Rightarrow J(s_0) = \begin{bmatrix} 2 & 0 & 0 \\ 1 & 1 & -1 \\ 2 & -1 & 0 \end{bmatrix}$, $F(s_0) = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}$

$s_{n+1} = s_n - J^{-1}(s_n) F(s_n) \overset{J(s_n)}{\Longleftrightarrow} s_{n+1} - s_n = -J^{-1}(s_n) \cdot F(s_n)$

$\Longleftrightarrow J(s_n)(s_{n+1} - s_n) = -F(s_n)$

$\Rightarrow J(s_1)(s_1 - s_0) = -F(s_0) \Longleftrightarrow \begin{bmatrix} 2 & 0 & 0 \\ 1 & 1 & -1 \\ 2 & -1 & 0 \end{bmatrix} \begin{bmatrix} x_1 - 1 \\ y_1 \\ z_1 \end{bmatrix} = - \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}$

$\Rightarrow \begin{cases} 2(x_1 - 1) = 0 \Rightarrow x_1 = 1 \\ x_1 - 1 + y_1 - z_1 = -1 \Rightarrow z_1 = 2 \\ 2\underbrace{(x_1 - 1)}_{=0} - y_1 = -1 \Rightarrow y_1 = 1 \end{cases} \Rightarrow s_1 = \begin{bmatrix} 1 \\ 1 \\ 2 \end{bmatrix}$

17. Ce îmbunătățiri aduc metodele moderne iterative pentru SEL? Ce relație au la bază și unde se caută soluția?

Metodele moderne iterative pt sisteme de ec. liniare (Jacobi, Gauss-Seidel, SOR) reduc timpul de rezolvare al sistemului, deoarece converg spre o soluție aproximativă mai rapid decât ar dura să găsim soluția

exactă ca alte metode (desc LU, eliminare gaussiană). Spre ex, un SEL care cu Jacobi s-ar rezolva cu 30 iterații, cu SOR, luând un ω bun, se poate rezolva într-o sg iterație.

Aceste metode au la bază subspațiul Krylov $Kr(A) = \text{Sp}\{b, Ab, A^2b \dots A^{n-1}b\}$, deci soluția se caută în comb. liniare ale elementelor din ↑.

18. Prezentați forma generală a metodelor iterative de soluționare a SEL. Particularizați pentru Jacobi, Gauss-Seidel și SOR. Ce condiție trebuie să satisfacă matricea de iterație a.î. metoda iterativă să conveargă?

Ca metoda iterativă să conveargă, deci să aibă sens utilizarea acestor metode, $\rho(G) < 1$, iar cu cât e mai mică cu atât mai rapid converge
↳ matricea de iterație. Fie sistemul $Ax = b$.

Matricea sistemului A se scrie ca $A = D - L - U$ (mat diag; mat inf Δ; mat sup Δ) $= N - P$, iar diferența dintre metode este dată de cum grupăm D, L și U în N și P.

1) Jacobi: $\begin{cases} N = D \\ P = L + U \end{cases}$ $\quad G_j = N^{-1}P \quad c_j = N^{-1}b \quad x_{k+1} = G_j x_k + c_j$

2) Gauss-Seidel: $\begin{cases} N = D - L \\ P = U \end{cases}$ $\quad G_{GS} = N^{-1}P \quad c_{GS} = N^{-1}b \quad x_{k+1} = G_{GS} x_k + c_{GS}$

3) SOR: $\begin{cases} N = D - \omega L \\ P = (1-\omega)L + \omega U \end{cases}$! $\quad G_{SOR} = N^{-1}P \quad c_{SOR} = \omega N^{-1}b \quad x_{k+1} = G_{SOR} x_k + c_{SOR}$

Forma algebrică: $x_{k+1} = (1-\omega)x_k + \omega GS$
↳ Gauss-Seidel la pasul k

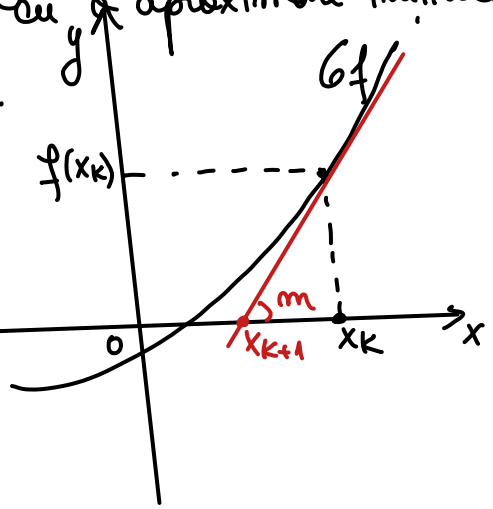
$1-\omega$ → mic ⇒ ω mare ⇒ sist explorează sol noi
→ mare ⇒ ω mic ⇒ sist ↓ soluții anterioare

ω = coeficientul de suprarelaxare

$\omega \in (0,2)$ → $\omega \in (0,1)$ converge lent
- $\omega = 1$ fix Gauss-Seidel
- $\omega \in (0,2)$ converge mai rapid

19. Explicați metoda Newton-Raphson.

Metoda Newton-Raphson (sau metoda tangentei) este o metodă de soluționare a ecuațiilor neliniare. Se începe cu o aproximare inițială x_0, fie găsită cu m. bisecției, fie luată ochiometric.



Prim pct $(x_k, f(x_k))$ se duce tangenta la Gf, aceasta intersectând axa Ox în pct $(x_{k+1}, 0)$, x_{k+1} = soluția la pasul $k+1$, iar m este panta, $m = f'(x_k)$

$$f'(x_k) = \frac{f(x_{k+1}) - f(x_k)}{x_{k+1} - x_k} = \frac{-f(x_k)}{x_{k+1} - x_k}$$

$$\Leftrightarrow \quad x_{k+1} - x_k = -\frac{f(x_k)}{f'(x_k)} \quad \Leftrightarrow \quad \boxed{x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}}$$

N-R nu converge dacă
- $f'(x_k) \approx 0$
- $|x_{k+1} - x_k| > 1$ (nu se mai poate neglija termenul 2 în pol. Taylor)

⊕ { are ordinul de convergență $R = 2$ deci e optimă
pt x_0 ales bine converge în 3-5 pași

⊖ { se apelează f de 2 ori
pt x_0 ales prost (departe de realitate) nu converge

20. Care este cea mai simplă formă pe care o putem obține cu transformări ortogonale Householder? În ce caz metoda Householder este cea mai eficientă? Explicați de ce transf. ortogonale Householder și Givens pot fi considerate factorizări.

Cea mai simplă formă pe care o putem obține cu transformări Hh este o matrice sup Δ, deoarece am făcut toate el de sub d. principală 0.

$$U = H_{m-1} \dots H_1 A$$

Metoda Householder are o complexitate de $O(n^3)$ și converge mai rapid decât Givens sau GPP; evită problemele de calcul cu zecimale de la GPP și față de Givens e mai eficientă când avem mai multe elemente pe aceeași coloană ce trebuie făcute zero deoarece le face deodată, față de Givens la care ar fi trebuit făcute pe rând.

Transf Hh și G pot fi considerate factorizări deoarece transformă sistemul într-unul mai simplu, mai ușor de rezolvat. Matricile H și G sunt ortogonale deci cu ajutorul acestor metode se descompune matricea sistemului A într-un produs de alte matrice.

21. Cum evităm calculul inversei matricei Jacobian în metoda Newton? Explicați detaliat.

Metoda Newton: $x_{n+1} = x_n - J^{-1}(x_n) \cdot F(x_n) \quad | - x_n$

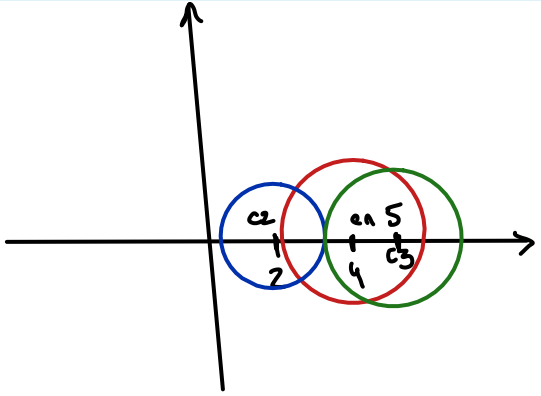
$$\Leftrightarrow J(x_n)\left| x_{n+1} - x_n = -J^{-1}(x_n) \cdot F(x_n)\right.$$

$$\Leftrightarrow J(x_n)(x_{n+1} - x_n) = -F(x_n)$$

Astfel evităm calculul inversei

22. Aprox cu 2 zecimale exacte, folosind o metodă numerică adecvată, aria reuniunii discurilor lui Gershgorin pt mat $A = \begin{bmatrix} 4 & 1 & 1 \\ 0 & 2 & 1 \\ -2 & 0 & 5 \end{bmatrix}$.

$$A = \begin{bmatrix} 4 & 1 & 1 \\ 0 & 2 & 1 \\ -2 & 0 & 5 \end{bmatrix} \Longrightarrow \begin{aligned} C_1&: |z-4| < 2 \Rightarrow z \in (2,6) \\ C_2&: |z-2| < 1 \Rightarrow z \in (1,3) \\ C_3&: |z-5| < 2 \Rightarrow z \in (3,7) \end{aligned}$$



2 zecimale ex $\Rightarrow$ tol $= 10^{-3}$

$A_{cerc} = \pi r^2$, dar raza e fixă $\Rightarrow$ calculăm π cu 2 zecimale exacte

$f(x) = \sin x$
$f'(x) = \cos x$

$f(\pi) = \sin \pi = 0$
$\Rightarrow \pi$ = pct de zero pt f

Începem cu aproximarea inițială $x_0 = 3$ $\xrightarrow{N-R}$

$$\Rightarrow x_1 = x_0 - \frac{f(x_0)}{f'(x_0)} = 3 - \frac{\sin 3}{\cos 3} = 3{,}1$$

$$x_2 = x_1 - \frac{f(x_1)}{f'(x_1)} = 3{,}1 - \frac{\sin 3{,}1}{\cos 3{,}1} = 3{,}14$$

Din principiul includerii și excluderii $\Rightarrow$

$$\Rightarrow A_U = A_{C_1} + A_{C_2} + A_{C_3} - A_{C_1 \cap C_2} - A_{C_1 \cap C_3}$$

23. Explicați cum ați aproxima cu n zecimale exacte, aria reuniunii discurilor Gershgorin pt o matrice (ce metode ați folosi și de ce)

Se folosește Newton-Raphson. Din moment $A = \pi r^2$, iar raza e dată de matrice, practic avem de calculat primele n zecimale ale lui π.

Luăm aprox inițială $x_0 = 3$ și apoi iterăm $x_{n+1} = x_n - \frac{\sin x}{\cos x}$ ($(\sin x)'$) până eroarea devine mai mică decât toleranța, $tol = 10^{-n-1}$

Apoi folosim principiul includerii și excluderii pt a determina aria.

24.* Alegeți mat coef A, dominant diagonală, și vectorul termenilor liberi b pentru sistemul de ecuații $Ax = b$. Comparați raza spectrală pt matricea de iterație Jacobi cu cea a mat. de iterație Gauss-Seidel folosind, poate, Metoda Puterii Inverse.

$\rho(G_j) > \rho(G\text{-}S)$ (Jacobi converge mai greu)

$A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ $\quad b = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ .....

25. Considerați A o matrice 3×3 pt care calculați o pereche (x, λ) de vector, valoare proprie. Alegeți $\alpha \in \mathbb{R}^*$ și $v \in \mathbb{R}^3$ un vector nenul, a.î. $v^T x = 1$. Calculați valorile proprii pt mat. Google $G = \alpha A + (1-\alpha)\lambda x v^T$.

nu e cazul

26. Aproximați cu 3 zecimale exacte, printr-o metodă numerică studiată pentru parțial, numărul $\sqrt{3}$.

$\downarrow tol = 10^{-4}$

Folosim metoda Newton-Raphson. Luăm $f(x) = x^2 - 3$

$\Rightarrow f(\sqrt{3}) = 0$ deci $\sqrt{3}$ pct de zero

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

$f'(x) = 2x$

Fie $x_0 = 2 \Rightarrow x_1 = 2 - \frac{2^2-3}{2\cdot 2} = 2 - \frac{1}{4} = \frac{7}{4} = 1.75$

$$x_2 = \frac{7}{4} - \frac{\left(\frac{7}{4}\right)^2 - 3}{2\cdot\frac{7}{4}} = \frac{7}{4} - \frac{\frac{49-48}{16}}{\frac{7}{2}} = \frac{7}{4} - \frac{1}{7\cdot 8} = 1{,}73$$

$x_3 = \ldots = 1{,}732$ $\quad$ STOP $(tol < 10^{-3})$

Bonus Aduceți o mat 3×3 de tip Householder la forma diagonală folosind transformate Givens. Se poate folosi o tehnică de simplificare a el. unei matrici folosind o matrice de tipul HG sau GH?

H este o matrice simetrică $\left(H = I_n - 2\frac{d \cdot d^t}{d^t \cdot d}\right) \Rightarrow$ dacă înmulțirea la stânga cu G face el $a_{ij} = 0$, atn la dreapta cu G^t face el $a_{ji} = 0$ astfel ducând matricea în una diagonală $D = G_k \cdots G_1 H G_1^t G_2^t \cdots G_k^t$

Exemplu:

$$H = I_3 - \frac{2 v v^T}{v^T \cdot v}$$

Fie $v = [0, 1, 2]^T$

$$\Rightarrow v \cdot v^T = \begin{bmatrix} 0 \\ 1 \\ 2 \end{bmatrix} \cdot [0, 1, 2] = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 2 \\ 0 & 2 & 4 \end{bmatrix}$$

$$v^T \cdot v = [0, 1, 2] \cdot \begin{bmatrix} 0 \\ 1 \\ 2 \end{bmatrix} = 1 + 4 = 5$$

$$\Rightarrow H = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} - \frac{2}{5} \cdot \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 2 \\ 0 & 2 & 4 \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 3/5 & -4/5 \\ 0 & -4/5 & -3/5 \end{bmatrix}$$

Vom elimina $(-4/5)$ de pe poziția $\{3,2\}$ și $\{2,3\}$

$$G_{32} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & c & -s \\ 0 & s & c \end{bmatrix}$$

$$c = \frac{a_{kk}}{r} \qquad s = -\frac{a_{ek}}{r}$$

$$r = \sqrt{\frac{9}{25} + \frac{16}{25}} = \sqrt{\frac{25}{25}} = 1 \quad \Rightarrow c = 3/5, \; s = +4/5$$

$$\Rightarrow G_{32} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 3/5 & -4/5 \\ 0 & +4/5 & 3/5 \end{bmatrix} \qquad G_{32}^T = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 3/5 & +4/5 \\ 0 & -4/5 & 3/5 \end{bmatrix}$$

$$\Rightarrow D = G_{32} \cdot H \, G_{32}^T = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 3/5 & 0 \\ 0 & 0 & -3/5 \end{bmatrix}$$

Coduri Matlab

1) Desc LU

-> Doolittle (L = unitriunghiulară)

```
function [L,U] = Doolittle (A)
    [n,n] = size(A);
    L = eye(n);
    U = zeros(n);
    for K=1:n
        for j = K:n
            s=0;
            for t = 1 : k-1
                s += L(k,t)*U(t,j);
            end
            U(K,j) = A(Kj)-s;
        end
        for i = k+1 :n
            s=0;
            for t= k+1 :n
                s+=L(i,t)*U(t,K);
            end
            L(i,K) = (A(i,K)-s)/U(K,K);
        end
    end
end
```

```
function [L, U] = Doolittle(A)
    [n, n] = size(A);
    L = eye(n);
    U = zeros(n);

    for k = [1 : n]
        % Partea I - calcularea valorilor u
        for j = [k : n]
            S = 0;
            for t = [1 : k-1]
                S += L(k,t) * U(t,j);
            end
            U(k,j) = A(k,j) - S;
        end
        % Partea II - calcularea valorilor l
        for i = [k+1 : n]
            S = 0;
            for t = [k+1 : n]
                S += L(i,t) * U(t,k);
            end
            L(i,k) = (A(i,k) - S) / U(k,k);
        end
    end
end
```

→ Crout (U unitriunghiulară)

```
function [L,U]= Crout (A)
    [n,n]=size(A);
    U=zeros(n);
    L=eye(n);
    for k=1:n
        for i = k:n
            s=0
            for t= 1:k-1
                s+= L(i,t)*U(t,k);
            end
            L(i,k)=A(i,k)-s;
        end
        for j=k+1:n
            s=0
            for t=k+1:n
                s+=L(k,t)*U(t,j);
            end
            U(k,j)=(A(k,j)-s)/L(k,k);
        end
    end
end
```

```
function [L, U] = Crout(A)
    [n, n] = size(A);
    L = eye(n);
    U = zeros(n);

    for k = [1 : n]
        % Partea I - calcularea valorilor l
        for i = [k : n]
            S = 0;
            for t = [1 : k-1]
                S += L(i,t) * U(t,k);
            end
            L(i,k) = A(i,k) - S;
        end
        % Partea II - calcularea valorilor u
        for j = [k+1 : n]
            S = 0;
            for t = [k+1 : n]
                S += L(k,t) * U(t,j);
            end
            U(k,j) = (A(k,j) - S) / L(k,k);
        end
    end
end
```

→ Cholesky

Fie A patratica, simetrica, pozitiv def ($x^t A x > 0 \ \forall x$)
≡ toate val proprii sunt poz

$U = L^t \Rightarrow A = L \cdot L^t$

```
function L = Cholesky(A)
    [n, n] = size(A);
    L = zeros(n);

    for i = [1 : n]
        % Valorile non-diagonale
        for j = [1 : i-1]
            S = 0;
            for k = [1 : j-1]
                S += L(i,k) * L(j,k);
            end
            L(i,j) = (A(i,j) - S) / L(j,j);
        end
        % Valorile diagonale
        S = 0;
        for j = [1 : i-1]
            S += L(i,j) * L(i,j);
        end
        L(i,i) = sqrt(A(i,i) - S);
    end
end
```

2) Factorizarea QR

→ Gram - Schmidt

```
function [Q, R] = gs(A)
    n = length(A);
    Q = R = zeros(n);
    for i = [1 : n]
        R(1:i-1, i) = Q(1:n, 1:i-1)' * A(1:n, i);
        y = A(1:n, i) - Q(1:n, 1:i-1) * R(1:i-1, i);
        R(i, i) = norm(y);
        Q(1:n, i) = y ./ R(i, i);
    end
end
```

→ Householder

$H = H^t$

```
function [Q, R] = householder(A)
    [m, n] = size(A);
    R = A;
    Q = eye(m);
    for i = [1 : n-1]
        a = R(i:m, i);
        r = norm(a);
        u_i = a - r * eye(m-i+1, 1);
        u_i = u_i / norm(u_i);

        R(i, i) = r; R(i+1:m, i) = 0;
        R(i:m,i+1:n) = R(i:m, i+1:n) - 2 * u_i * (u_i' * R(i:m,i+1:n));

        Q(1:m, i:m) = Q(1:m, i:m) - Q(1:m, i:m) * 2 * u_i * u_i';
    end
end
```

→ Givens

```
function [Q, R] = givens(A)
    [n, m] = size(A);
    Q = eye(m);

    for j = [1 : min(n-1, m)]
        for k = [j + 1 : n]
            rho = sqrt(A(j,j)^2 + A(k,j)^2);
            c = A(j,j) / rho;
            s = -A(k,j) / rho;

            aux = [c -s; s c] * [A(j, j:m); A(k, j:m)];
            A(j, j:m) = aux(1, :);
            A(k, j:m) = aux(2, :);

            aux = [c -s; s c] * [Q(j, 1:n); Q(k, 1:n)];
            Q(j, 1:n) = aux(1, :);
            Q(k, 1:n) = aux(2, :);
        end
    end

    Q = Q';
    R = A;
end
```

jau

```
function [Q,R] = hh(A)
  [m,n] = size(A)
  R = A
  Q = eye(m)
  for i = 1 : min(m-1, n)
    V = zeros(m)
    V(i:m) = A(i:m, i)
    V(i) = V(i) - sign(V(i)) * norm(V)
    norm = norm(V)
    H = eye(m) - 2 * V * V.' / norm
    Q = Q * H
    A = H * A
  end
  R = A
end
```

```
function [Q,R] = givens(A)
  [m,n] = size(A)
  Q = eye(m)
  for i = 1 : n
    for j = i+1 : m
      if A(j,i) != 0
        G = eye(m)
        r = norm([A(j,i); A(i,i)])
        G(j,i) = -A(j,i) / r
        G(i,j) = -G(j,i)
        G(i,i) = G(j,j) = A(i,i) / r
        Q = Q * G'
        A = G * A
      end   (la if)
    end
  end
  Q = Q.'
end
```

3) Eliminare Gaussiană

→ G

```
function [A, b] = G(A, b)
    [m, n] = size(A);
    Ae = [A, b];
    for p = [1 : n-1]
        if (A(p ,p) == 0)
            continue;
        end
        for l = p+1 : n
            arg = Ae(l, p) / Ae(p, p);
            Ae(l, :) = Ae(l, :) - arg * Ae(p, :);
        end
    end
    A = Ae(:, 1:n);
    b = Ae(:, n+1);
end
```

→ GPP

```
function [A, b] = GPP(A, b)
    [m, n] = size(A);
    Ae = [A, b];
    for p = [1 : n-1]
        % Partea noua (GPP):
        [piv, l_piv] = max(abs(Ae(p:n, p)));
        l_piv = l_piv + p - 1;
        temp = Ae(l_piv, :);
        Ae(l_piv,:) = Ae(p, :);
        Ae(p, :) = temp;

        % Algoritmul G:
        if (Ae(p ,p) == 0)
            continue;
        end
        for l = p+1 : n
            arg = Ae(l, p) / Ae(p, p);
            Ae(l, :) = Ae(l, :) - arg * Ae(p, :);
        end
    end
    A = Ae(:, 1:n);
    b = Ae(:, n+1);
end
```

→ GPPS

```
function [A, b] = GPPS(A, b)
    [m, n] = size(A);
    Ae = [A, b];
    for p = [1 : n]
        S(p) = max(abs(A(p,:)));
    end
    for p = [1 : n-1]
        % Partea de GPPS:
        [piv, l_piv] = max(abs(Ae(p:n, p)) ./ S(:,p:n)');
        l_piv = l_piv + p - 1;
        temp = Ae(l_piv, :);
        Ae(l_piv,:) = Ae(p, :);
        Ae(p, :) = temp;
        temp = S(l_piv);
        S(l_piv) = S(p);
        S(p) = temp;

        % Algoritmul G:
        if (Ae(p ,p) == 0)
            continue;
        end
        for l = p+1 : n
            arg = Ae(l, p) / Ae(p, p);
            Ae(l, :) = Ae(l, :) - arg * Ae(p, :);
        end
    end
    A = Ae(:, 1:n);
    b = Ae(:, n+1);
end
```

→ GPT

```
function [A, b] = GPT(A, b)
    [m, n] = size(A);
    Ae = [A, b];
    for p = [1 : n-1]
        % Partea noua (GPT):
        [piv, l_piv] = max(abs(Ae(p:n, p)));
        [piv, c_piv] = max(piv);
        l_piv = l_piv + p - 1;
        l_piv = l_piv(c_piv);
        c_piv = c_piv + p - 1;
        temp = Ae(l_piv, :);
        Ae(l_piv, :) = Ae(p, :);
        Ae(p, :) = temp;
        temp = Ae(:, c_piv);
        Ae(:, c_piv) = Ae(:, p);
        Ae(:, p) = temp;
        % Algoritmul G:
        if (Ae(p ,p) == 0)
            continue;
        end
        for l = p+1 : n
            arg = Ae(l, p) / Ae(p, p);
            Ae(l, :) = Ae(l, :) - arg * Ae(p, :);
        end
    end
    A = Ae(:, 1:n);
    b = Ae(:, n+1);
end
```

→ Thomas

```
function x = Thomas(a, b, c, d)
    n = length(d);
    a = [0; a];
    c(1) = c(1) / b(1);
    d(1) = d(1) / b(1);
    for i = [2 : n-1]
        temp = b(i) - a(i) * c(i-1);
        c(i) = c(i) / temp;
        d(i) = (d(i) - a(i) * d(i-1)) / temp;
    end
    d(n) = (d(n) - a(n) * d(n-1)) / (b(n) - a(n) * c(n-1));
    x(n) = d(n);
    for i = [n-1 : -1 : 1]
        x(i) = d(i) - c(i) * x(i+1);
    end
    x = x';
end
```

4) Metode iterative de rez a SEL

→ Jacobi

```
function x = jacobi(A, b, x0, tol, max_iter)
    n = length(A);
    x = x0;

    for k = [1 : max_iter]
        for i = [1 : n]
            val_sum = (A(i,:) * x0(:)) - (A(i,i) * x0(i));
            x(i) = (b(i) - val_sum) / A(i,i);
        end
        if norm(x - x0) < tol
            break
        end
        x0 = x
    end
end
```

→ Gauss - Seidel

```
function x = gauss_seidel(A, b, x0, tol, max_iter)
    n = length(A);
    x = x0;

    for k = [1 : max_iter]
        for i = [1 : n]
            val_sum = (A(i,1:i-1) * x(1:i-1)) + (A(i,i+1:n) * x0(i+1:n));
            x(i) = (b(i) - val_sum) / A(i,i);
        end
        if norm(x - x0) < tol
            break
        end
        x0 = x;
    end
end
```

→ SOR

```
function x = sor(A, b, x0, omega, tol, max_iter)
    n = length(A);
    x = x0;

    for k = [1 : max_iter]
        for i = [1 : n]
            val_sum = (A(i,1:i-1) * x(1:i-1)) + (A(i,i+1:n) * x0(i+1:n));
            x(i) = (1-omega) * x0(i) + omega * (b(i) - val_sum) / A(i,i);
        end
        if norm(x - x0) < tol
            break
        end
        x0 = x
    end
end
```

5) M. pt ec neliniare

-> M. bisectiei

```
function c = metoda_bisectiei(a, b, f, tol, max_iter)
  if f(a) == 0
    c = a;
    return;
  elseif f(b) == 0
    c = b;
    return;
  end
  for k = [1 : max_iter]
    c = (a + b) / 2;
    if c - a < tol
      return
    end
    val_a = f(a);
    val_c = f(c);
    if val_c == 0
      return
    elseif val_a * val_c < 0
      b = c;
    else
      a = c;
    end
  end
end
```

-> M. tangentei (Newton-Raphson)

```
function x = metoda_tangentei(x_0, P, err, max_iter)
  dP = polyder(P);
  x = x_0;
  for k = [1 : max_iter]
    y0 = polyval(P, x);
    if abs(y0) < err
      return
    end
    dy0 = polyval(dP, x);
    x -= y0 / dy0;
  end
end
```

-> M. secantei

```
function x2 = metoda_secantei(x0, x1, f, err, max_iter)
  y0 = f(x0);
  if abs(y0) < err
    x2 = x0;
    return
  end

  y1 = f(x1);
  if abs(y1) < err
    x2 = x1;
    return
  end

  for k = [1 : max_iter]
    x2 = x1 - y1 * (x1 - x0) / (y1 - y0);
    x0 = x1; y0 = y1;
    x1 = x2; y1 = f(x1);
    if abs(y1) < err
      return
    end
  end
end
```

→ Gradient desc

```
function x = descendent_gradient(A, b, tol, max_iter)
    r = b;
    x = zeros(size(b));
    i = 1;

    while i <= max_iter
        if norm(r) < tol
            break;
        end

        ar = A * r;
        alpha = (r' * r) / (r' * ar);
        x = x + alpha * r;
        r = r - alpha * ar;

        i = i + 1;
    end
end
```

→ Gradient conjugat

```
function x = conjugate_gradient(A, b, tol, max_iter)
    r = b;
    p = r;
    x = zeros(size(b));
    i = 1;

    while i <= max_iter
        ap = A * p;
        pap = p' * ap;
        rr = r' * r;
        alpha = rr / pap;
        x = x + alpha * p;
        r = r - alpha * ap;

        if norm(r) < tol
            break;
        end

        beta = (r' * r) / rr;
        p = r + beta * p;
        i = i + 1;
    end
end
```

Pivotare partial

$\left\{\begin{matrix} LU \\ QR \end{matrix}\right.$ → $O(n^3)$ $\quad Ax = b \Leftrightarrow x = A^{-1}b$

$\det(A) \rightarrow O(n!)$

$LU \Rightarrow LUx = b \Rightarrow \begin{cases} Ly = b \rightarrow \text{sist inf } \Delta & O(n^2) \\ Ux = y \rightarrow \text{sist sup } \Delta \end{cases}$

↳ Crout
↳ Doolittle
↳ Cholesky

$QR \Rightarrow QRx = b \Leftrightarrow Rx = Q^* b$

↳ Givens
↳ Householder

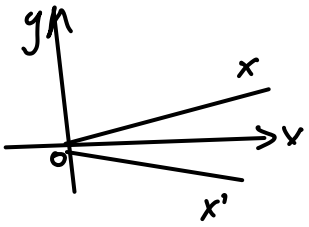
$\underbrace{Q_M \dots Q_1}_{Q} A = R \Rightarrow A = Q^t R$

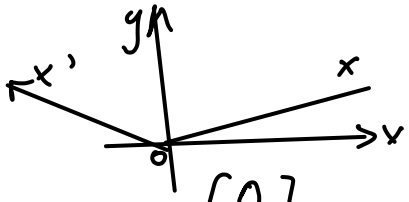
Householder $A_1 = \begin{bmatrix} 2 & 4 & 5 \\ 1 & -1 & 1 \\ 2 & 1 & -1 \end{bmatrix}$

$v = \begin{bmatrix} 2 \\ 1 \\ 2 \end{bmatrix}$, $d = \begin{bmatrix} 2 - \text{sgn}(2) \cdot \|v\| \\ 1 \\ 2 \end{bmatrix} = \begin{bmatrix} -1 \\ 1 \\ 2 \end{bmatrix}$

$H_1 = I - 2\frac{dd^t}{d^t d}$

De ce $-\text{sgn}$?





$$H_1 A = \begin{bmatrix} 3 & 3 & 3 \\ 0 & -3 & -3 \\ 0 & 3 & 3 \end{bmatrix} \qquad v = \begin{bmatrix} 0 \\ 3 \end{bmatrix}, \ \|v\| = 3 \Rightarrow d = \begin{bmatrix} 0 \\ -3 \\ 3 \end{bmatrix}$$

calculez $\Rightarrow H_2 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$

Givens $\qquad A = \begin{bmatrix} 0 & 1 & 2 \\ 3 & 2 & 0 \\ 4 & 1 & 5 \end{bmatrix} \rightarrow (i,j) = (3,1)$

$$\Rightarrow G_{31} = \begin{bmatrix} c & 0 & -s \\ 0 & 1 & 0 \\ s & 0 & c \end{bmatrix}$$

$$c = \frac{0}{\sqrt{0^2+4^2}} = 0, \quad s = \frac{-4}{\sqrt{0^2+4^2}} = -1$$

$$\begin{bmatrix} c & -s \\ s & c \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} r \\ 0 \end{bmatrix}$$

$$\Rightarrow G_{31} = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \end{bmatrix} \qquad G_{32}$$

$$G_{21} = \begin{bmatrix} c & -s & 0 \\ s & c & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

EXAM Cum aduci o mat Hh la diag folosind Givens ?

H = simetrică

înmulțim la stânga cu G^t

GHA → faci mai întâi pe col 1 zero și apoi G pe (3,2)

AHG → faci mai întâi pe linia 1 zero și apoi G pe (2,3)

T. Schur: $A = Q^t U Q$ $\quad A = P^{-1} J P$

J — blocuri Jordan $J_i = \begin{bmatrix} \lambda_i & 1 & 0 & \\ 0 & \lambda_i & 1 & \\ & & \ddots & \end{bmatrix}$

- $A = \begin{bmatrix} 3 & 2 & 102 \\ 2 & -3 & 101 \\ 1 & \alpha & 2 \end{bmatrix}$ $\quad b = \begin{bmatrix} 101 \\ 96 \\ 2 \end{bmatrix}$

$102 - 3 + 2 = 101$

$101 - 2 - 3 = 96$

$2 - 1 + \alpha = 2 \Rightarrow \alpha = 1$ $\quad \Rightarrow x = \begin{bmatrix} -1 \\ 1 \\ 1 \end{bmatrix}$

GPPS pe $A = \begin{bmatrix} 3 & 2 & 102 \\ 2 & -3 & 101 \\ 1 & 1 & 2 \end{bmatrix}$

$s_1 = \left| \frac{3}{102} \right|$

$s_2 = \left| \frac{2}{101} \right|$

$s_3 = \left| \frac{1}{2} \right|$ cel mai mare

$L_1 \leftrightarrow L_3 \begin{bmatrix} 1 & 1 & 2 & 2 \\ 2 & -3 & 101 & 96 \\ 1 & 1 & 2 & 101 \end{bmatrix}$ calculez — - ---

Iterative SEL

$A = M - N \Rightarrow (M-N)x = b \Rightarrow Mx = Nx + b$

$\Rightarrow x = M^{-1} N x + M^{-1} b$

$A = D - L - U$

$G = M^{-1} N$ mat iteratie

$c = M^{-1} b$ vect iteratie

$x^{(m+1)} = G x^{(m)} + c$ $\quad$ cond conv: $|\rho(G)| < 1$

-> Jacobi $\begin{cases} M = D \\ N = L + U \end{cases}$

-> Gauss-Seidel $\begin{cases} M = D - L \\ N = U \end{cases}$

if (norm($x^{(n)}, x^{(n-1)}$ (, inf/e/norma cerută)) < tol)
break;

Mat să fie dom diagonală $|a_{ii}| \geq \sum_{i \neq j} a_{ij}$

SEL

Gradient conjugat

~ pct fixe $x^2 - x - 1 = 0$ $f(x) = x$

$\Rightarrow x = \sqrt{x+1}$
$\Rightarrow x = x^2 - 1$
$\Rightarrow x = \frac{1}{x-1}$

$k|x-y| > |f(x) - f(y)|$
$k < 1 \Rightarrow f$ contracție
$\Leftrightarrow \frac{|f(x)-f(y)|}{|x-y|} < k$
$|f'(c)| < k < 1$

$\left(\sqrt{x+1}\right)' = \frac{1}{2\sqrt{x+1}} < 1$ pt $\forall x \in (0, \infty)$

$x^{(n+1)} = f(x^{(n)})$

Aia cu ales de sisteme neliniare

$v^{(n+1)} = F(v^{(n)})$

Pct fixe: $\begin{cases} x^2 - 2y = 2 \\ y - \ln(x) = 1 \end{cases}$

$\Rightarrow \begin{cases} x = \sqrt{2+2y} \\ y = \ln x + 1 \end{cases}$

$\left(\sqrt{2+2y}\right)' = \frac{1}{\sqrt{2+2y}} < 1 \Leftrightarrow \sqrt{2+2y} > 1$
$\Rightarrow 2y > -1$
$\Rightarrow y > -\frac{1}{2}$

$\Rightarrow (1 + \ln x)' = \frac{1}{x} \Rightarrow x > 1$

$D = (1, \infty)$

M. Newton

$\begin{cases} f_1(x_1 \dots x_n) = 0 \\ \dots \\ f_n(x_1 \dots x_n) = 0 \end{cases}$

$x^{(n+1)} = x^{(n)} - \frac{f(x^{(n)})}{f'(x^{(n)})}$

$x^{(n+1)} = x^{(n)} - J^{-1}(x_n) F(x_n)$

$\Rightarrow J(x_m)\left(x^{(m+1)} - x^{(m)}\right) = -F(x_m)$

→ Discurile lui Gershgorin

$$\begin{bmatrix} 1 & 2 & -3 \\ 4 & 5 & 6 \\ 1 & 8 & -9 \end{bmatrix} \rightarrow \begin{array}{l} |z-1| < 5 \\ |z-5| < 10 \\ |z+9| < 15 \end{array}$$

$Adisc = \pi r^2$ → o rotiliu

↙ trb să calculăm π

GRADIENT CONJUGAT

v, u conj $\Leftrightarrow v^t A u = 0$

$\langle v, u \rangle_A = 0$

$Ax^* = b \Rightarrow b = \sum \lambda_i A p_i$

$$p_j^t b = \sum \lambda_i p_j^t A p_i = \lambda_j p_j^t A p_j \Rightarrow \lambda_j = \frac{p_j^t b}{p_j^t A p_j}$$

$x^{(m+1)} = x^{(m)} - \lambda^{(m)} p^{(m)}$

$x^{(0)} = 0$

$r^{(0)} = b$

$x^{(1)} = \lambda_1 b$

$p^{(1)} = b - \lambda_1 Ab$

$x^{(2)} = \lambda_1 \delta_1 + \lambda_2 \cdots$

$\Rightarrow x^{(m)} \in \text{sph}\{b, Ab, A^2 b \cdots\}$

$=$ subsp Krylov

Gradientul

$Ax=b$! A semipozitiv definită $x^t A x \geq 0$

x^* sol sist $\Leftrightarrow$ x^* minimizează $g(x) = \langle x, Ax \rangle - 2\langle x, b \rangle$

$x_{n+1} = x_n + t_{n+1} v_{n+1}$

Gr conjugat

$v^t A u = 0 \Leftrightarrow \langle v, u \rangle_A = 0$ ($\leftarrow$ GS modificat pe A)

n vectori A-ortogonali $\begin{cases} \langle v_i, v_i \rangle = 1 \\ \langle v_i, v_j \rangle = 0 \quad i \neq j \end{cases}$

$$t^k = \frac{\langle r^{k-1}, r^{k-1} \rangle}{\langle v^k, Av^k \rangle} \qquad r^{k+1} = r^k - t^k A v^k$$

$$s^k = \frac{\langle r^k, r^k \rangle}{\langle r^{k-1}, r^{k-1} \rangle} \qquad v^{k+1} = r^k + s^k \cdot v^k$$

$r_0 = b - A x_0$, $v_0 = r_0$

$Kr(A) = sp\{b, Ab, A^2 b \dots\}$

Sistem Newton

$$\begin{cases} f_1 : \frac{x}{5} - \frac{y^2}{16} = 0 \\ f_2 : \frac{x^3}{3} + \frac{y}{4} = 0 \end{cases}$$

$\frac{\partial f_1}{\partial x} = \frac{1}{5}$ $\quad \frac{\partial f_1}{\partial y} = \left| \frac{-y}{5} \right|$

$\frac{1}{5} + \left| \frac{y}{5} \right| < 1 \Leftrightarrow |y| < 4 \Rightarrow y \in (-4, 4)$

$\frac{\partial f_2}{\partial x} = x^2$ $\quad \frac{\partial f_2}{\partial y} = \frac{1}{4}$ $\Rightarrow$

$x^2 + \frac{1}{4} < 1 \Leftrightarrow x^2 < \frac{3}{4} \Rightarrow x \in \left(\frac{-\sqrt{3}}{2}, \frac{\sqrt{3}}{2} \right)$

$\Rightarrow D = \left(\frac{-\sqrt{3}}{2}, \frac{\sqrt{3}}{2} \right)$

$x_{n+1} = x_n - J^{-1}(x_n) \cdot f(x_n)$

$\Leftrightarrow \int(x_n)(x_{n+1}-x_n) = -F(x_n)$

$x_0 = \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$

$\begin{bmatrix} \frac{1}{5} & -\frac{1}{5} \\ 1 & \frac{1}{4} \end{bmatrix}\begin{bmatrix} x_1-1 \\ y_1-1 \end{bmatrix} = \begin{bmatrix} -\frac{1}{10} \\ -\frac{7}{12} \end{bmatrix}$ . . . . . .

Cova pct fixe

$f_1\begin{cases} x+y^2=5 \Rightarrow x=5-y^2 \Rightarrow (5-y^2)'=-2y \Rightarrow |-2y|<1 \\ \Rightarrow y\in(-\frac{1}{2},\frac{1}{2}) \end{cases}$

$f_2\begin{cases} x^3+2y=6 \Rightarrow y=\frac{6-x^3}{2} \Rightarrow \frac{-3x^2}{2} \Rightarrow x\in(-\sqrt{\frac{2}{3}},\sqrt{\frac{2}{3}}) \end{cases}$

$\Rightarrow D=(-\sqrt{\frac{2}{3}},\sqrt{\frac{2}{3}}) \quad x_0=\begin{bmatrix} 0 \\ 0 \end{bmatrix}$

$v_1 = F(x_0)$

$x_1 = \begin{bmatrix} 5 \\ 3 \end{bmatrix}$ (normal) sau $x_1 = \begin{bmatrix} 5 \\ 3-\frac{5^3}{2} \end{bmatrix}$ (ca G-S)

- 1) H $\longrightarrow$ diag cu Givens
2) G $\longrightarrow$ diag cu H

1) $H = I_3 - 2\frac{dd^t}{\|d\|^2} \Rightarrow$ Hortogonale și sim

$H = \begin{bmatrix} \ddots & & x \\ & \ddots & \\ x & & \ddots \end{bmatrix}$

$H' = G_{n1}\cdot H\Big|^t$

$\Leftrightarrow (H')^t = H^t G_{n1}^t$

$\Leftrightarrow H = H G_{n1}^t \quad \dots$

$\Rightarrow D = G\cdot H\cdot G^t$

$Ax = b$

x^* – soluția sistemului $\Rightarrow x^*$ minimizează funcția

$$g(x) = \langle x, Ax \rangle - 2\langle x, b \rangle$$

$$x_{k+1} = x_k + t_k A v_k$$

$$t^k = \frac{\langle r_{k-1}, r_{k-1} \rangle}{\langle v_k, A v_k \rangle} \qquad r^{k+1} = r^k - t^k A v_k$$

$$s^k = \frac{\langle r_k, r_k \rangle}{\langle r_{k-1}, r_{k-1} \rangle} \qquad v^{k+1} = r_k + s_k v_k$$

A – SPD $\quad x^T A x \geq 0 \; \forall x \neq 0$